

# Indeksy, optymalizator

## Lab1

---

### Imiona i nazwiska:

- Paweł Moćko
  - Katarzyna Wesołowska
- 

Celem ćwiczenia jest zapoznanie się z planami wykonania zapytań (execution plans), oraz z budową i możliwością wykorzystaniem indeksów.

Swoje odpowiedzi wpisuj w miejsca oznaczone jako:

---

Wyniki:

-- ...

---

Ważne/wymagane są komentarze.

Zamieść kod rozwiązania oraz zrzuty ekranu pokazujące wyniki

- dołącz kod rozwiązania w formie tekstowej/źródłowej
- najlepiej plik .md
  - ewentualnie sql

Zwróć uwagę na formatowanie kodu

## Oprogramowanie - co jest potrzebne?

Do wykonania ćwiczenia potrzebne jest następujące oprogramowanie

- MS SQL Server
- SSMS - SQL Server Management Studio
  - ewentualnie inne narzędzie umożliwiające komunikację z MS SQL Server i analizę planów zapytań
- przykładowa baza danych AdventureWorks2017.

Oprogramowanie dostępne jest na przygotowanej maszynie wirtualnej

## Przygotowanie

Stwórz swoją bazę danych o nazwie lab1.

```
create database lab1  
go  
  
use lab1  
go
```

## Część 1

---

Celem tej części ćwiczenia jest zapoznanie się z planami wykonania zapytań (execution plans) oraz narzędziem do automatycznego generowania indeksów.

## Dokumentacja/Literatura

Przydatne materiały/dokumentacja. Proszę zapoznać się z dokumentacją:

- <https://docs.microsoft.com/en-us/sql/tools/dta/tutorial-database-engine-tuning-advisor>
- <https://docs.microsoft.com/en-us/sql/relational-databases/performance/start-and-use-the-database-engine-tuning-advisor>
- <https://www.simple-talk.com/sql/performance/index-selection-and-the-query-optimizer>
- <https://blog.quest.com/sql-server-execution-plan-what-is-it-and-how-does-it-help-with-performance-problems/>

Operatory (oraz reprezentujące je piktogramy/lkonki) używane w graficznej prezentacji planu zapytania opisane są tutaj:

- <https://docs.microsoft.com/en-us/sql/relational-databases/showplan-logical-and-physical-operators-reference>

Wykonaj poniższy skrypt, aby przygotować dane:

```
select * into [salesorderheader]
from [adventureworks2017].sales.[salesorderheader]
go

select * into [salesorderdetail]
from [adventureworks2017].sales.[salesorderdetail]
go
```

## Zadanie 1 - Obserwacja

---

Wpisz do MSSQL Managment Studio (na razie nie wykonuj tych zapytań):

```
-- zapytanie 1
select *
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate = '2008-06-01 00:00:00.000'
go

-- zapytanie 1.1
select *
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate = '2013-01-28 00:00:00.000'
go

-- zapytanie 2
select orderdate, productid, sum(orderqty) as orderqty,
       sum(unitpricediscount) as unitpricediscount, sum(linetotal)
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
group by orderdate, productid
having sum(orderqty) >= 100
go

-- zapytanie 3
select salesordernumber, purchaseordernumber, duedate, shipdate
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate in ('2008-06-01', '2008-06-02', '2008-06-03', '2008-06-04', '2008-
06-05')
go

-- zapytanie 4
select sh.salesorderid, salesordernumber, purchaseordernumber, duedate, shipdate
from salesorderheader sh
```

```
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where carriertrackingnumber in ('ef67-4713-bd', '6c08-4c4c-b8')
order by sh.salesorderid
go
```

Włącz dwie opcje: **Include Actual Execution Plan** oraz **Include Live Query Statistics**:

Teraz wykonaj poszczególne zapytania (najlepiej każde analizuj oddzielnie). Co można o nich powiedzieć? Co sprawdzają? Jak można je zoptymalizować?

#### Wyniki:

#### Komentarze:

- Zapytanie 1:
  - zapytanie łączy tabele salesorderheader z salesorderdetail, i zwraca rekordy gdzie data zamówienia to 01.06.2008r
  - w tym przypadku zapytanie zwraca 0 wierszy
  - w planie wykonania widać Table Scan oraz Hash Match (Inner Join), czyli SQL Server przeszukuje całe tabele mimo braku wyników
  - optymalizacją byłoby utworzenie indeksu na kolumnie orderdate w tabeli salesorderheader oraz na kolumnie salesorderid w tabeli salesorderdetail, co ograniczyłoby skanowanie tabel
- Zapytanie 1.1:
  - zapytanie ma taką samą konstrukcję jak zapytanie 1, ale dotyczy daty 2013-01-28
  - w tym przypadku zapytanie zwraca wiersze, a koszt obu zapytań jest bardzo podobny: dla zapytania 1 wynosi około 2.45, a dla zapytania 1.1 około 2.47
  - propozycja optymalizacji byłaby taka sama jak w zapytaniu 1
  - przykład pokazuje, że bez indeksu SQL Server przeszukuje całą tabelę zarówno wtedy, gdy rekordy istnieją, jak i wtedy, gdy wynik końcowy jest pusty
- Zapytanie 2
  - zapytanie łączy tabele salesorderheader i salesorderdetail, grupuje dane po orderdate i productid oraz liczy sumy wartości
  - sprawdza agregację po złączeniu i filtruje grupy warunkiem having sum(orderqty) >= 100
  - w planie widać skanowanie obu tabel, operator Hash Match oraz operatory Równoległość, czyli SQL Server uznał, że zapytanie jest na tyle kosztowne, że warto wykonać je równoległe.
  - w planie wykonania widać, że operator działał w 8 wątkach, co oznacza, że optymalizator podzielił pracę między kilka wątków, aby przyspieszyć wykonanie zapytania.
  - optymalizacją byłoby dodanie indeksów na salesorderid oraz kolumnach używanych w grupowaniu i sumowaniu, żeby ograniczyć skanowanie tabel i przyspieszyć agregację
- Zapytanie 3:
  - zapytanie łączy tabele salesorderheader i salesorderdetail oraz zwraca odpowiednie kolumny dla zamówień z podanych 5 dni

- w tym przypadku zapytanie nie zwraca żadnych wierszy
  - w planie nadal widać skanowanie obu tabel oraz Hash Match (Inner Join), czyli SQL Server przeszukuje całe tabele mimo pustego wyniku
  - optymalizacją byłoby utworzenie indeksu na orderdate w salesorderheader oraz indeksu na salesorderid w salesorderdetail
- Zapytanie 4:
    - zapytanie łączy tabele salesorderheader i salesorderdetail oraz zwraca wybrane kolumny dla rekordów, w których carriertrackingnumber ma jedną z podanych wartości
    - w planie widać skanowanie obu tabel, Hash Match (Inner Join) oraz Sort, czyli SQL Server najpierw przeszukuje dane, potem łączy tabele, a na końcu sortuje wynik po salesorderid
    - optymalizacją byłoby utworzenie indeksu na carriertrackingnumber oraz salesorderid w tabeli salesorderdetail, ponieważ warunek filtrowania dotyczy tej tabeli

---

## Zadanie 2 - Dobór indeksów / optymalizacja

---

Do wykonania tego ćwiczenia potrzebne jest narzędzie SSMS

Zapytania 1, 2, 3, 4 z poprzedniego zadania

```
select *
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate = '2008-06-01 00:00:00.000'
go

-- zapytanie 2
select orderdate, productid, sum(orderqty) as orderqty,
       sum(unitpricediscount) as unitpricediscount, sum(linetotal)
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
group by orderdate, productid
having sum(orderqty) >= 100
go

-- zapytanie 3
select salesordernumber, purchaseordernumber, duedate, shipdate
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate in ('2008-06-01', '2008-06-02', '2008-06-03', '2008-06-04', '2008-
06-05')
go

-- zapytanie 4
select sh.salesorderid, salesordernumber, purchaseordernumber, duedate, shipdate
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where carriertrackingnumber in ('ef67-4713-bd', '6c08-4c4c-b8')
```

```
order by sh.salesorderid  
go
```

Zaznacz wszystkie zapytania, i uruchom je w **Database Engine Tuning Advisor**:

Sprawdź zakładkę **Tuning Options**, co tam można skonfigurować?

---

#### Wyniki:

Komentarz: W zakładce Tuning Options ustawia się, w jaki sposób DTA (Database Engine Tuning Advisor) ma analizować podane zapytania i czego ma szukać podczas optymalizacji. Można tam określić czas analizy, to czy narzędzie ma proponować indeksy, widoki indeksowane lub partycjonowanie, a także czy ma zachować już istniejące struktury w bazie.

---

Użyj **Start Analysis**:

Zaobserwuj wyniki w **Recommendations**.

Przejdź do zakładki **Reports**. Sprawdź poszczególne raporty. Główną uwagę zwróć na koszty i ich poprawę:

Zapisz poszczególne rekomendacje:

Uruchom zapisany skrypt w Management Studio.

Opisz, dlaczego dane indeksy zostały zaproponowane do zapytań:

---

#### Wyniki: Komentarze:

- Wyniki działania DTA pokazano na końcu raportu: [Rysunek 1](#)
- W zakładce Recommendations narzędzie DTA zaproponowało kilka nowych indeksów dla tabel salesorderheader i salesorderdetail.
- Szacowana poprawa wydajności wyniosła 73%, co oznacza, że zaproponowane indeksy powinny wyraźnie obniżyć koszt wykonania analizowanych zapytań.

Uwaga! [Rysunek 2](#) Pomimo wielokrotnych prób nie udało się poprawnie odczytać zawartości zakładki Reports w Database Engine Tuning Advisor. W dalszej części również zakładka Recommendations przestała działać poprawnie, dlatego opis rekomendowanych indeksów został przygotowany na podstawie wcześniej zachowanego zrzutu ekranu.

Komentarz:

- DTA zaproponował indeksy dla tabel salesorderheader i salesorderdetail, ponieważ analizowane zapytania często używają kolumn OrderDate, SalesOrderID, ProductID i CarrierTrackingNumber. Są to kolumny używane w filtrowaniu, złączeniach, grupowaniu i sortowaniu.
- Indeksy na OrderDate i SalesOrderID mają przyspieszyć zapytania 1 i 3, ponieważ tam filtrowanie odbywa się po dacie, a złączenie po SalesOrderID. Dzięki temu można ograniczyć pełne skanowanie tabel.

- Indeksy związane z ProductID oraz kolumnami używanymi w sumach zostały zaproponowane dla zapytania 2, ponieważ to zapytanie wykonuje grupowanie i agregację. Z kolei indeks na CarrierTrackingNumber został zaproponowany dla zapytania 4, bo właśnie po tej kolumnie odbywa się filtrowanie.

---

Sprawdź jak zmieniły się Execution Plany. Opisz zmiany:

---

#### Wyniki:

Próba częściowego odtworzenia indeksów na bazie zachowanego załącznika, aby móc chociaż pokazać różnice w planach.

```
create nonclustered index ix_salesorderheader_orderdate_salesorderid
on salesorderheader(orderdate, salesorderid);
go

create nonclustered index ix_salesorderheader_salesorderid_cover
on salesorderheader(salesorderid)
include (salesordernumber, purchaseordernumber, duedate, shipdate);
go

create nonclustered index ix_salesorderdetail_salesorderid
on salesorderdetail(salesorderid);
go

create nonclustered index ix_salesorderdetail_carriertrackingnumber_salesorderid
on salesorderdetail(carriertrackingnumber, salesorderid);
go

create nonclustered index ix_salesorderdetail_salesorderid_productid
on salesorderdetail(salesorderid, productid)
include (orderqty, unitpricediscount, linetotal);
go

--zapytania
select *
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate = '2008-06-01 00:00:00.000';
go

select orderdate, productid, sum(orderqty) as orderqty,
       sum(unitpricediscount) as unitpricediscount, sum(linetotal)
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
group by orderdate, productid
having sum(orderqty) >= 100;
go
```

```
select salesordernumber, purchaseordernumber, duedate, shipdate
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate in ('2008-06-01', '2008-06-02', '2008-06-03', '2008-06-04', '2008-
06-05');
go

select sh.salesorderid, salesordernumber, purchaseordernumber, duedate, shipdate
from salesorderheader sh
inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where carriertrackingnumber in ('ef67-4713-bd', '6c08-4c4c-b8')
order by sh.salesorderid;
go
```

#### Komentarze:

- Zapytanie 1:
  - zamiast skanowania całych tabel pojawiło się Index Seek oraz Nested Loops.
  - SQL Server najpierw szybko wyszukuje pasujące rekordy po orderdate, a potem dobiera odpowiadające im wiersze z drugiej tabeli, co jest bardziej efektywne niż wcześniejszy Table Scan i Hash Match
- Zapytanie 2:
  - plan nadal korzysta z równoległości, ale zamiast skanować całe tabele bezpośrednio, SQL Server odczytuje dane z utworzonych indeksów
  - operator agregacji i filtr po HAVING, więc zapytanie nadal pozostaje kosztowne, ale indeksy lepiej wspierają złączenie i grupowanie niż wcześniejszy pełny Table Scan
- Zapytanie 3:
  - Zmiana jest analogiczna jak w zapytaniu 1: zamiast skanowania całych tabel pojawiły się Index Seek oraz Nested Loops.
  - SQL Server korzysta teraz z indeksu do szybkiego znalezienia pasujących dat, a następnie dobiera odpowiadające im rekordy z drugiej tabeli.
- Zapytanie 4:
  - zamiast skanowania tabel pojawiły się Index Seek, Nested Loops oraz Sort.
  - SQL Server najpierw wyszukuje pasujące rekordy po carriertrackingnumber, następnie łączy je z tabelą salesorderheader, a na końcu sortuje wynik po salesorderid

Podsumowanie: indeksy mają duży wpływ na sposób wykonania zapytań przez SQL Server. Po ich dodaniu optymalizator przestał w wielu przypadkach skanować całe tabele i zaczął korzystać z Index Seek, czyli szybkiego wyszukiwania tylko potrzebnych danych.

---

## Część 2

---

Celem ćwiczenia jest zapoznanie się z różnymi rodzajami indeksów oraz możliwością ich wykorzystania

## Dokumentacja/Literatura

Przydatne materiały/dokumentacja. Proszę zapoznać się z dokumentacją:

- <https://docs.microsoft.com/en-us/sql/relational-databases/indexes/indexes>
- <https://docs.microsoft.com/en-us/sql/relational-databases/sql-server-index-design-guide>
- <https://www.simple-talk.com/sql/performance/14-sql-server-indexing-questions-you-were-too-shy-to-ask/>
- <https://www.sqlshack.com/sql-server-query-execution-plans-examples-select-statement/>

## Zadanie 3 - Indeksy klastrowane I nieklastrowane

Skopiuj tabelę **Customer** do swojej bazy danych:

```
select * into customer from adventureworks2017.sales.customer
```

Wykonaj analizy zapytań:

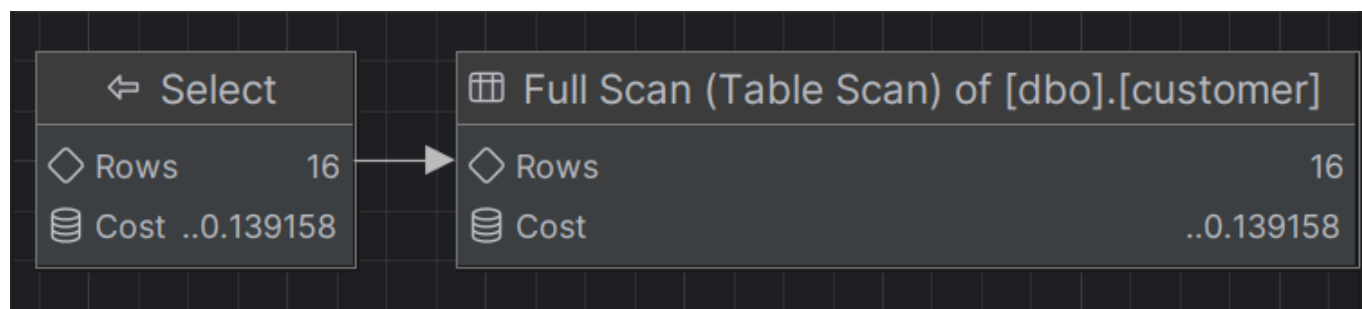
```
select * from customer where storeid = 594  
  
select * from customer where storeid between 594 and 610
```

Zanotuj czas zapytania oraz jego koszt koszt:

### Wyniki przed indeksowaniem

Dla pierwszego zapytania:

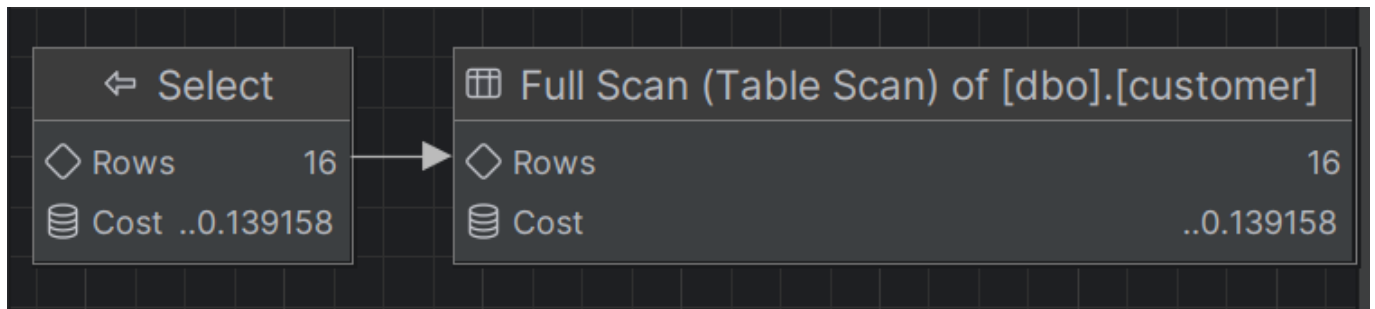
- czas wykonania: 8 ms
- koszt: 0.139158
- plan wykonania:



Dla drugiego zapytania:

- czas wykonania: 9 ms

- koszt: 0.139158
- plan wykonania:



Koszt wykonania obu zapytań jest identyczny. Dzieje się tak dlatego, że tabela **Customer** nie ma zdefiniowanego indeksu dla kolumny **StoreID**.

**Obserawcje** W przypadku braku indeksu, przeszukiwane są wszystkie wiersze bazy danych. Możliwa byłaby optymalizacja dla warunku **StoreID = 594**, jeśli byłoby nałożone ograniczenie **UNIQUE** - jednak nie ma to miejsca.

Dodaj indeks:

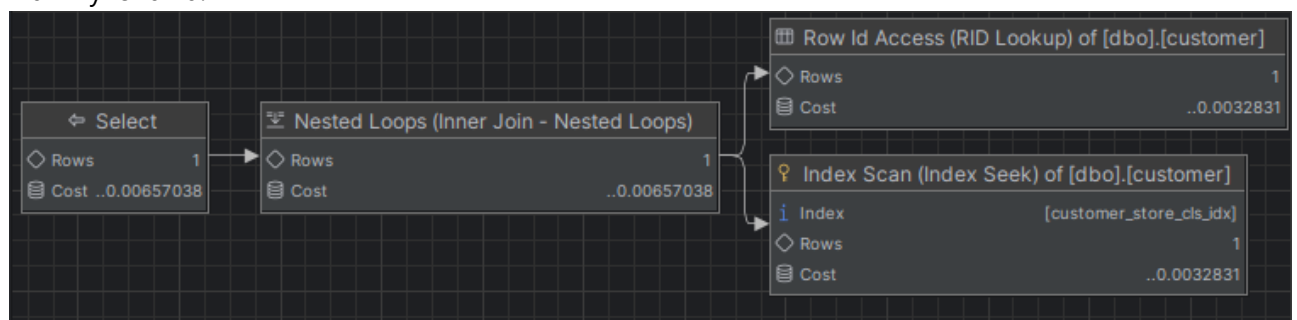
```
create index customer_store_cls_idx on customer(storeid)
```

Wyniki po indeksowaniu

Jak zmienił się plan i czas? Czy jest możliwość optymalizacji?

**Po dodaniu indeksu otrzymaliśmy następujące wyniki:** Dla pierwszego zapytania:

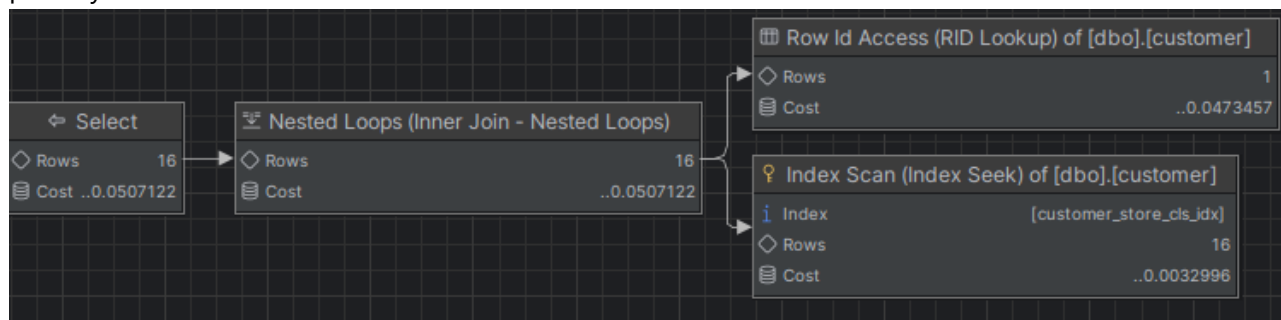
- czas wykonania: 5ms
- koszt: 0.00657038
- Plan wykonania:



Dla drugiego zapytania:

- czas wykonania: 4ms
- koszt: 0.0507122

- plan wykonania:



### Obserwacje:

- czas wykonania oraz koszt zapytania został znacznie zredukowany dla obu zapytań
- koszt zapytania różni się między zapytaniami - co nie miało wcześniej miejsca.
- SQL Server może teraz rozróżnić pracę wymaganą do pozyskania jednego wiersza a zakresu wierszy.
- Operacja **Full Scan**, która znajdowała się w planach zapytań w przypadku braku indeksacji została zastąpiona operacjami o znacznie mniejszym koszcie.

Korzyści pod względem bardziej wydajnego przeszukiwania są konsekwencją indeksacji. Wówczas wyszukiwanie wierszy sprowadza się do wyszukiwania wartości w zbalansowanym drzewie (B-Tree), gdzie możliwe jest ustalenie lokalizacji wierszy o danej wartości kolumny **storeID**.

Dodaj indeks klastrowany:

```
create clustered index customer_store_cls_idx on customer(storeid)
```

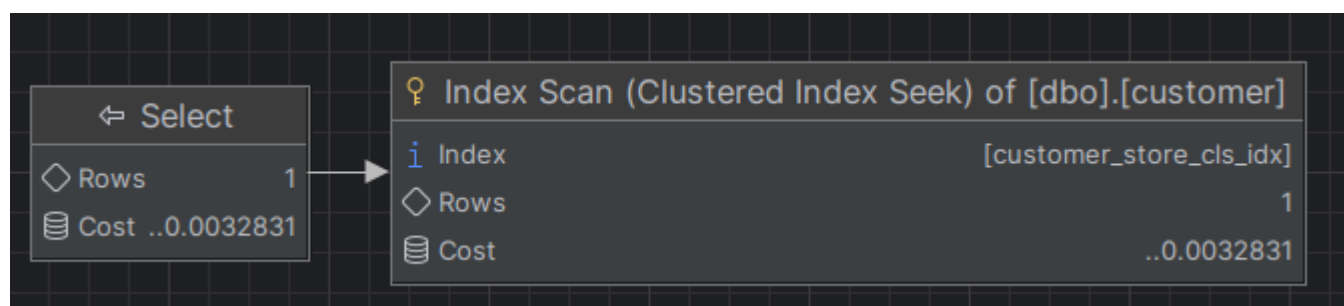
Czy zmienił się plan/koszt/czas? Skomentuj dwa podejścia w wyszukiwaniu krotek.

Przed dodaniem indeksu klastrowanego, usunęliśmy istniejący już indeks:

```
DROP INDEX customer_store_cls_idx ON customer;
```

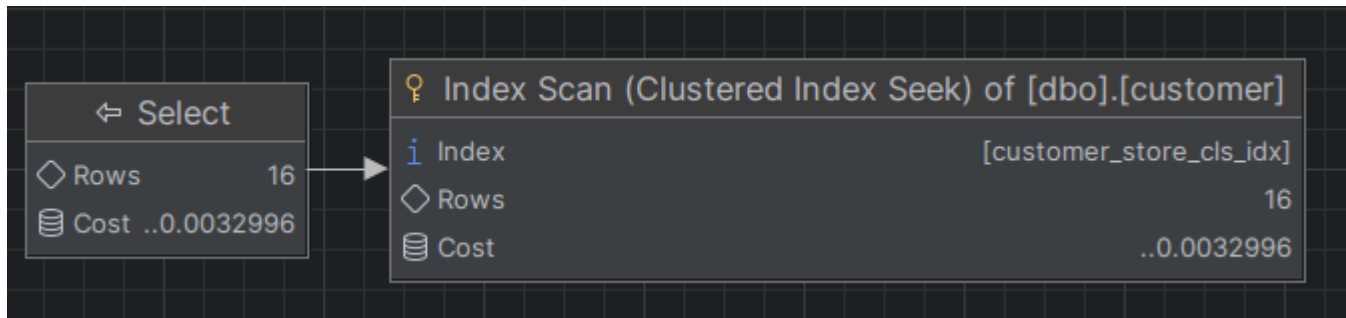
**Po dodaniu indeksu klastrowanego:** Dla pierwszego zapytania:

- czas wykonania: 2ms
- koszt: 0.0032831
- plan zapytania:



Dla drugiego zapytania

- czas wykonania: 2ms
- koszt: 0.0032996
- plan zapytania:



### Obserwacje:

- Koszt wykonania ponownie zmalał. Podobnie z czasem wykonania.
- Plan zapytania również uległ zmianie. Jeden z kroków został pominięty.
- Koszt obu zapytań jest równy

Różnica w działaniu indeksów:

- dla indeksu klastrowanego wiersze tabeli są sortowane<sup>1</sup>
- drugi rodzaj indeksu wykorzystuje strukturę danych, która pozwala na szybkie wyszukanie wg wartości klucza indeksacji. Ta struktura również zawiera wskaźnik do wiersza tabeli, który jest później zwracany. Ze względu na tę różnicę w działaniu, pozyskanie wierszów wg **StoreID** jest mniej kosztowne w przypadku użycia indeksu klastrowanego.

## Zadanie 4 - dodatkowe kolumny w indeksie

Celem zadania jest porównanie indeksów zawierających dodatkowe kolumny.

Skopiuj tabelę **Address** do swojej bazy danych:

```
select * into address from adventureworks2017.person.address
```

W tej części będziemy analizować następujące zapytanie:

```
select addressline1, addressline2, city, stateprovinceid, postalcode
from address
where postalcode between '98000' and '99999'
```

```
create index address_postalcode_1
on address (postalcode)
include (addressline1, addressline2, city, stateprovinceid);
```

```
go

create index address_postalcode_2
on address (postalcode, addressline1, addressline2, city, stateprovinceid);
go
```

Czy jest widoczna różnica w planach/kosztach zapytań? [^]

- w sytuacji gdy nie ma indeksów
- przy wykorzystaniu indeksu:
  - address\_postalcode\_1
  - address\_postalcode\_2

Jeśli tak to jaka?

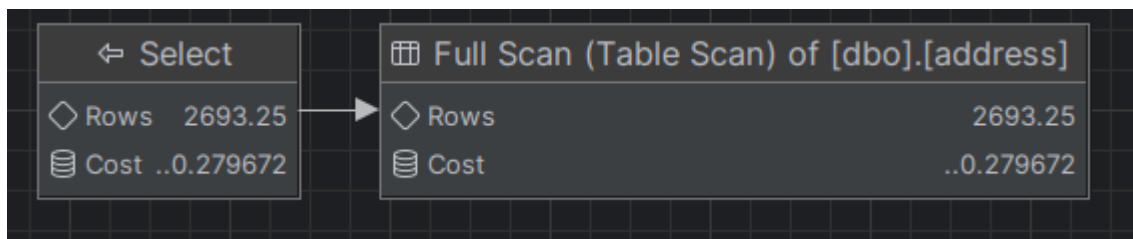
Aby wymusić użycie indeksu użyj `WITH(INDEX(Address_PostalCode_1))` po `FROM`

```
select addressline1, addressline2, city, stateprovinceid, postalcode
from address WITH(INDEX(Address_PostalCode_1))
where postalcode between '98000' and '99999'

select addressline1, addressline2, city, stateprovinceid, postalcode
from address WITH(INDEX(Address_PostalCode_2))
where postalcode between '98000' and '99999'
```

Brak indeksu

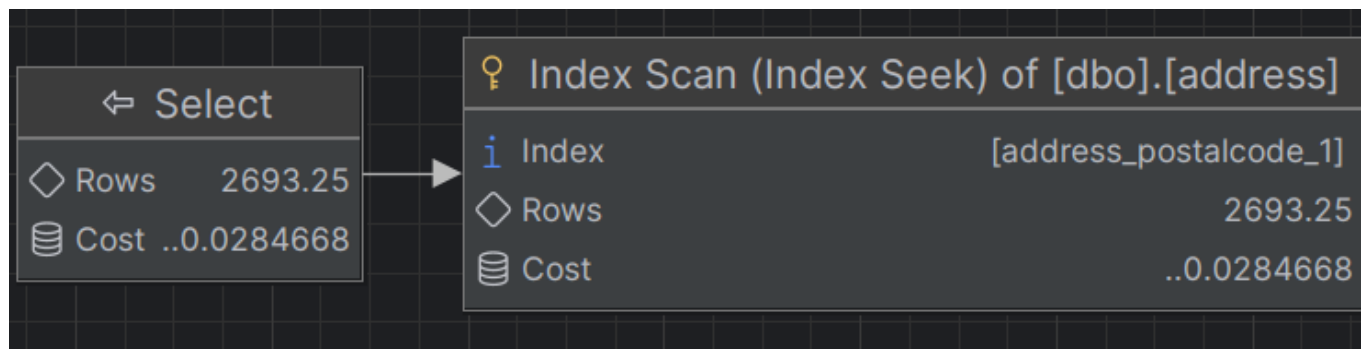
**Czas wykonania:** 7ms **Koszt:** 0.279672 **Plan wykonania**



Podobnie jak wcześniej - sprawdzane są wszystkie wiersze tabeli.

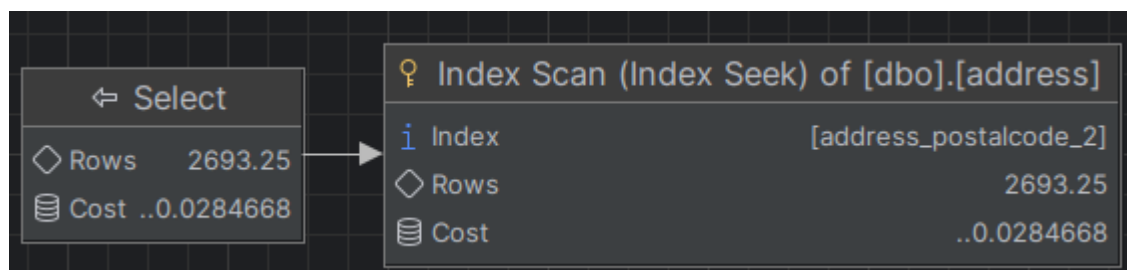
Indeks `address_postalcode_1`

**Czas wykonania:** 7ms **Koszt:** 0.0284668 **Plan wykonania**



Indeks `address_postalcode_2`

**Czas wykonania:** 5ms **Koszt:** 0.0284668 **Plan wykonania**



Dla obu indeksów uzyskaliśmy ten sam koszt.

Sprawdź rozmiar Indeksów:

```
select i.name as indexname, sum(s.used_page_count) * 8 as indexsizekb
from sys.dm_db_partition_stats as s
inner join sys.indexes as i on s.object_id = i.object_id and s.index_id =
i.index_id
where i.name = 'address_postalcode_1' or i.name = 'address_postalcode_2'
group by i.name
go
```

Który jest większy? Jak można skomentować te dwa podejścia do indeksowania? Które kolumny na to wpływają?

Wynik zapytania:

| indexname            | indexsizekb |
|----------------------|-------------|
| address_postalcode_1 | 1784        |
| address_postalcode_2 | 1808        |

Pierwszy indeks zajmuje mniej pamięci. Jest to spowodowane wykorzystaniem słowa kluczowego `INCLUDE` zamiast włączania pozostałych kolumn do klucza indeksowania. Dzieje się tak, ponieważ informacja o pozostałych kolumnach jest zawarta wyłącznie w liściach drzewa użytego na potrzeby indeksacji. W przypadku wykorzystania większej ilości kluczy indeksacji, każdy węzeł takiego drzewa zawiera informację dot. każdej kolumny - co zwiększa ilość wymaganej pamięci.

Dodanie informacji o dodatkowych kolumnach w indeksie pozwala na przyspieszenie wyszukiwania wartości. W powyższych przykładach wszystkie wymagane dane dla przeszukiwanych wierszy można było pozyskać bez potrzeby dostępu do tabeli z wykorzystaniem wskaźnika jak w zadaniu 4.

## Zadanie 5 - kolejność atrybutów

Skopiuj tabelę **Person** do swojej bazy danych:

```
select businessentityid
      ,persontype
      ,namestyle
      ,title
      ,firstname
      ,middlename
      ,lastname
      ,suffix
      ,emailpromotion
      ,rowguid
      ,modifieddate
into person
from adventureworks2017.person.person
```

Wykonaj analizę planu dla trzech zapytań:

```
select * from [person] where lastname = 'Agbonile'

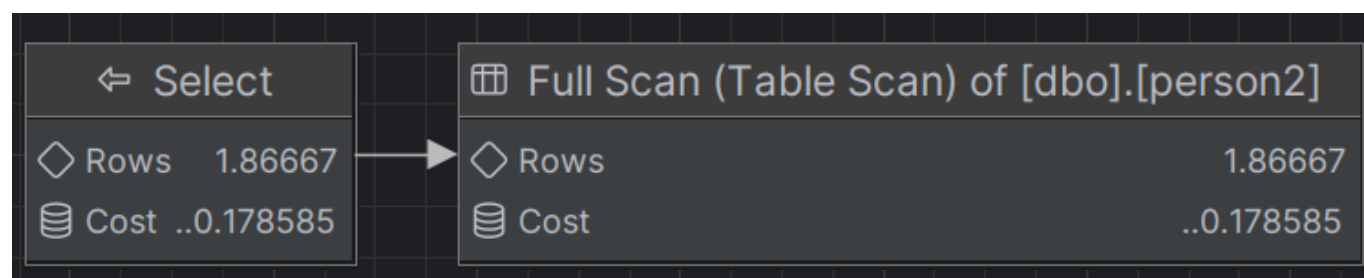
select * from [person] where lastname = 'Agbonile' and firstname = 'Osarumwense'

select * from [person] where firstname = 'Osarumwense'
```

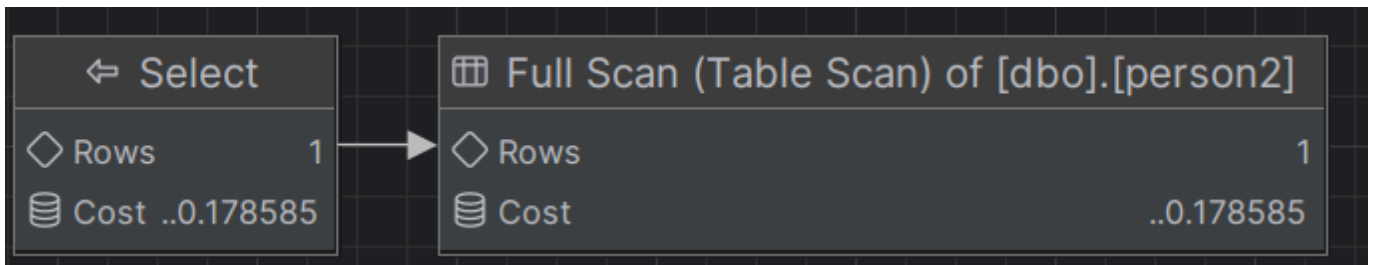
Co można o nich powiedzieć?

Wyniki:

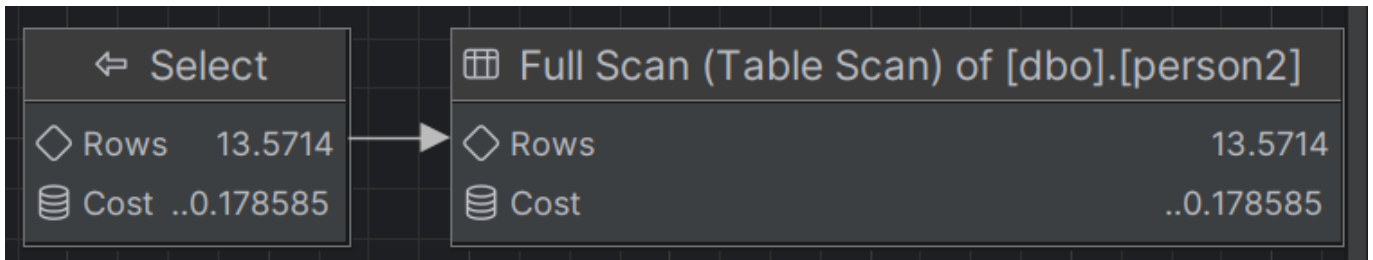
### Zapytanie 1



### Zapytanie 2



### Zapytanie 3



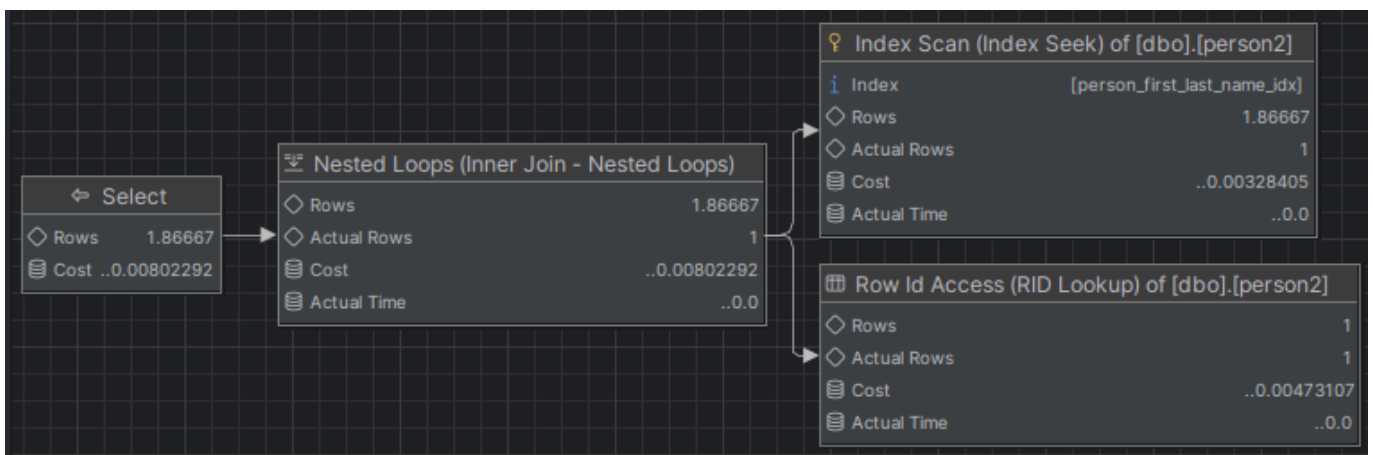
Dla każdego z trzech zapytań sprawdzane są wszystkie wiersze tabeli. Różnią się jedynie przewidywaną liczbą sprawdzonych wierszy.

Przygotuj indeks obejmujący te zapytania:

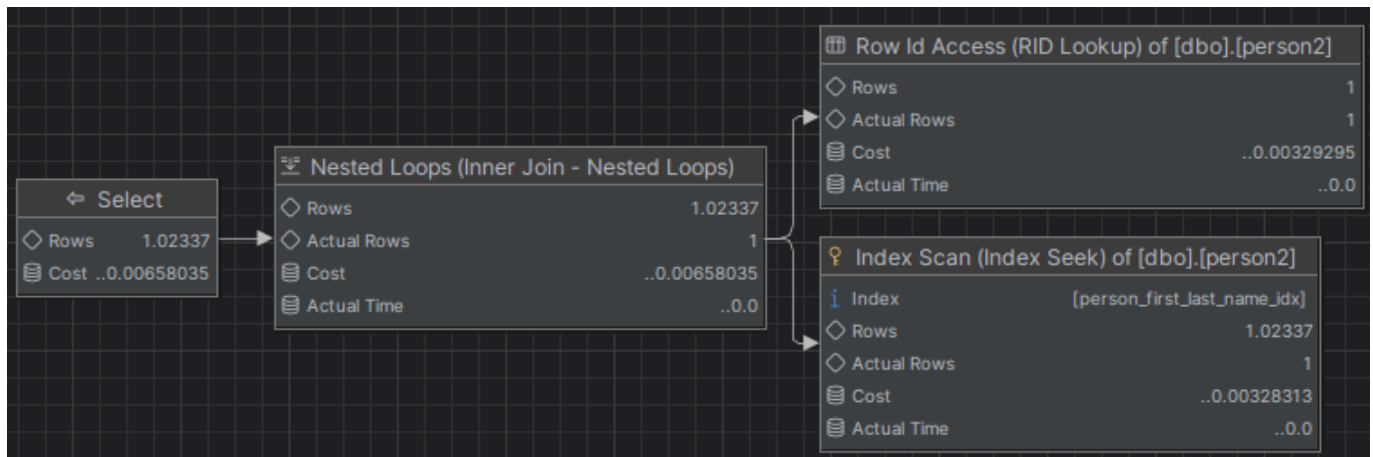
```
create index person_first_last_name_idx
on person(lastname, firstname)
```

Sprawdź plan zapytania. Co się zmieniło?

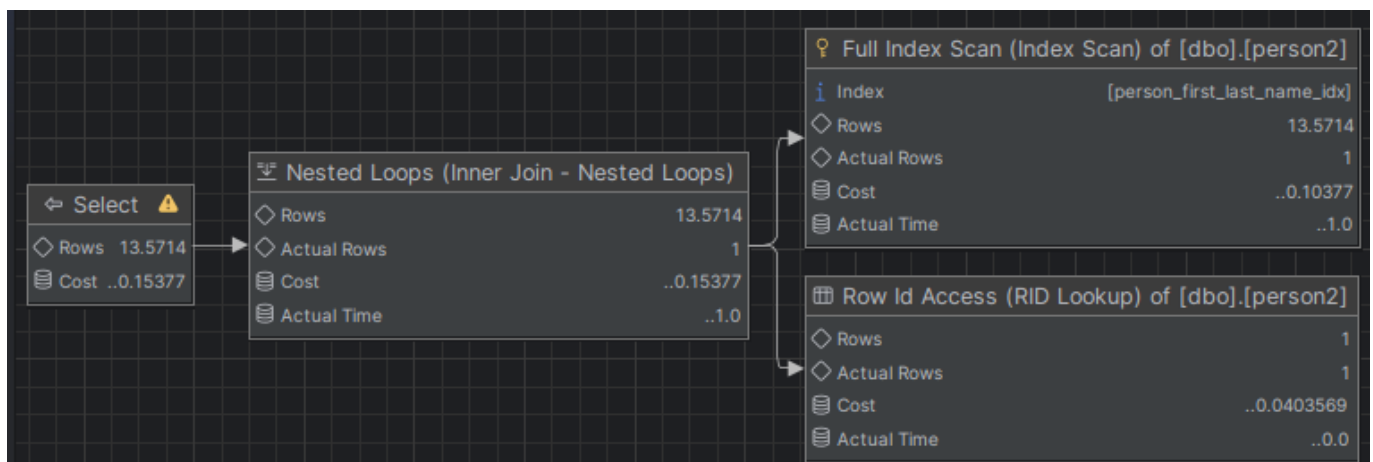
### Zapytanie 1



### Zapytanie 2



### Zapytanie 3



Na podstawie analizy planów obserwujemy, że:

- koszt dla zapytań 1. oraz 2. jest równy.
- koszt dla zapytania 3. jest znacznie wyższy od pozostałych

Różnica kosztu między zapytaniami, gdzie filtrujemy wg wartości `lastname` jest istotna. Dzieje się tak, ponieważ kolejność kolumn przy tworzeniu indeksu złożonego jest istotna. W tym przypadku wartości kolumn nie są sortowane globalnie wg kolumny `firstname`, lecz lokalnie (w obrębie wierszy o tej samej wartości `lastname`)<sup>2</sup>.

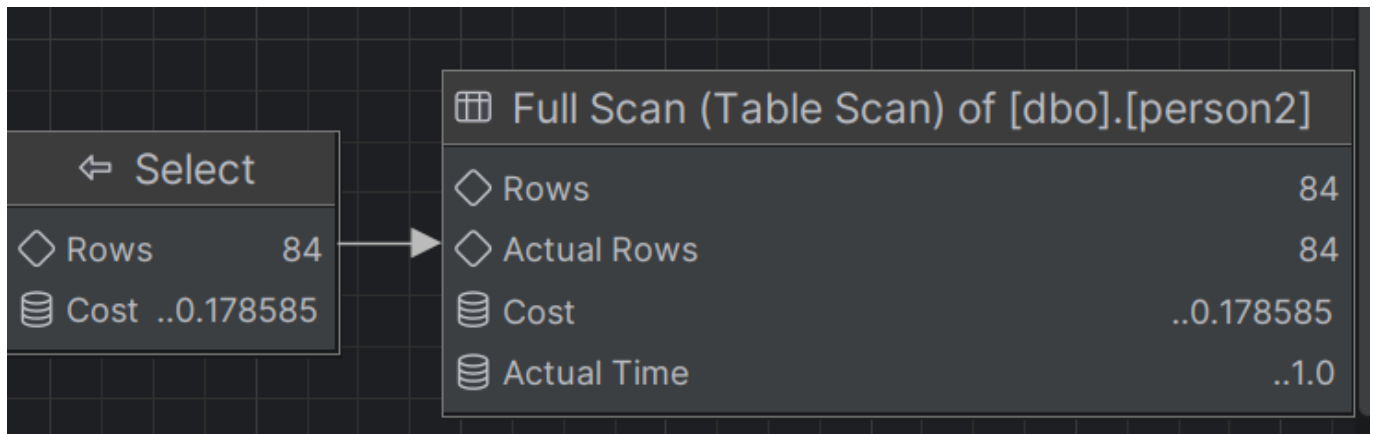
Przeprowadź ponownie analizę zapytań tym razem dla parametrów: `FirstName = 'Angela' LastName = 'Price'`. (Trzy zapytania, różna kombinacja parametrów).

Czym różni się ten plan od zapytania o `'Osarumwense Agbonile'`. Dlaczego tak jest?

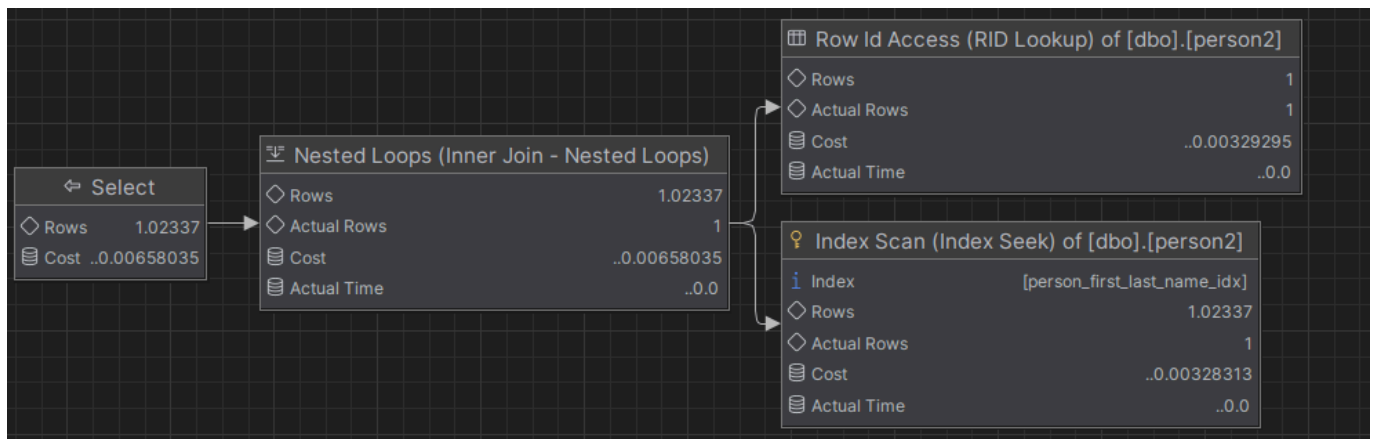
Wyniki:

Wykonaliśmy trzy zapytania oraz plany wykonania:

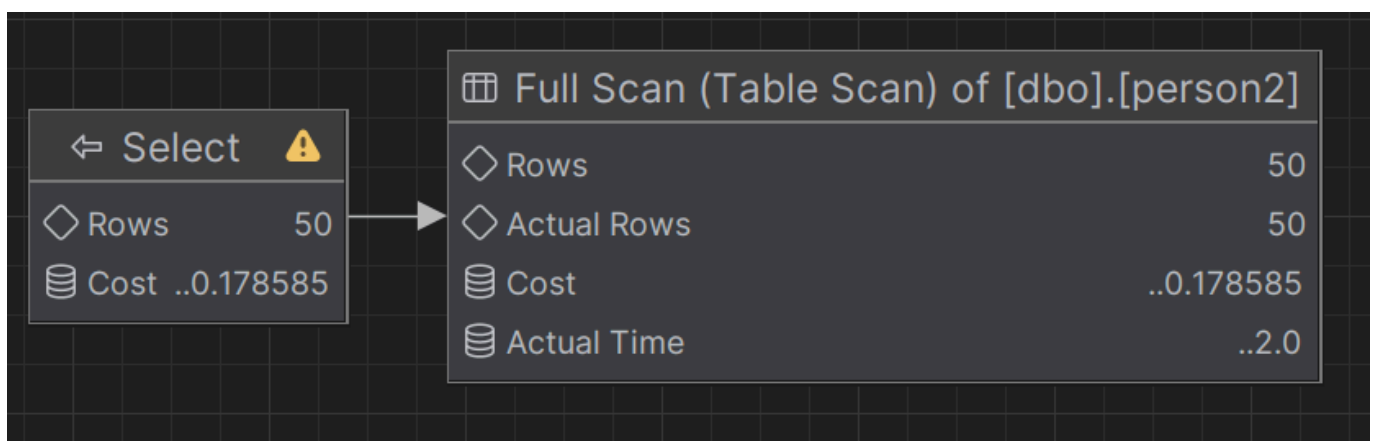
```
SELECT *
FROM person2
WHERE lastname = 'Price'
```



```
SELECT *
FROM person2
WHERE firstname = 'Angela'
AND lastname = 'Price'
```



```
SELECT *
FROM person2
WHERE firstname = 'Angela'
```



Dla pierwszego oraz trzeciego zapytania wykonane zostanie przeszukanie z wykorzystaniem wszystkich wierszy tabeli. Dzieje się tak ponieważ tym zapytaniom przydzielany jest większy koszt w przypadku

wykorzystania indeksu, niż w przypadku przeszukania całej tabeli. W celu zweryfikowania tej hipotezy wykonaliśmy poniższe zapytania:

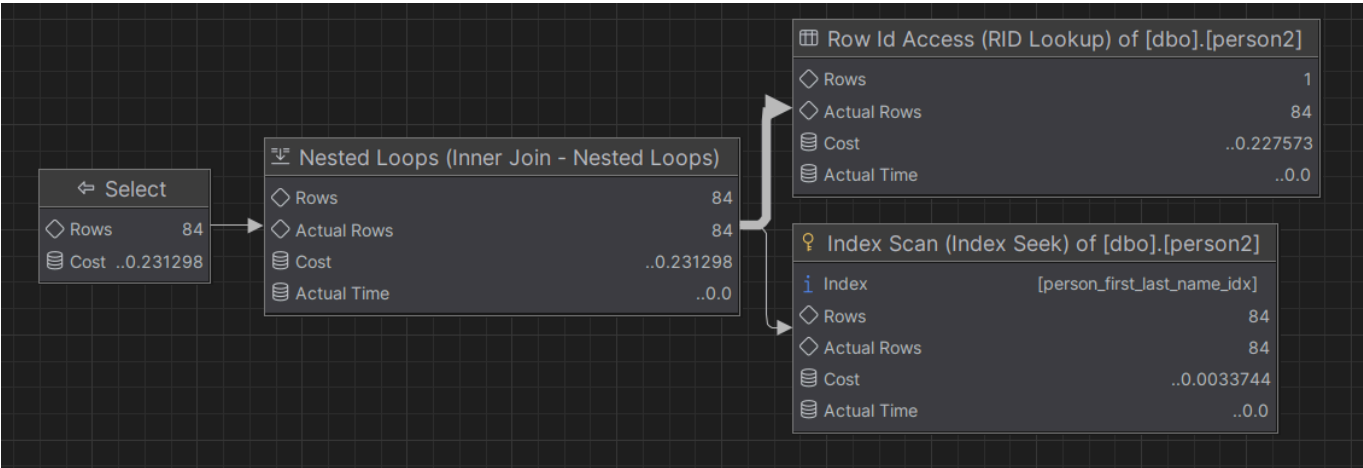
```
SELECT *
FROM person2
    WITH ( INDEX ( person_first_last_name_idx ))
WHERE lastname = 'Price'

SELECT *
FROM person2
    WITH ( INDEX ( person_first_last_name_idx ))
WHERE firstname = 'Angela'
```

I otrzymaliśmy koszty:

| warunek zapytania          | koszt bez<br>specyzowania indeksu | koszt dla sprecyzowanego<br>indeksu |
|----------------------------|-----------------------------------|-------------------------------------|
| WHERE lastname = 'Price'   | 0.178585                          | 0.231298                            |
| WHERE firstname = 'Angela' | 0.178585                          | 0.258187                            |

Rzeczywiście wykorzystanie indeksu wiąże się z większą wartością kosztu.



Powyższy plan wykonania dla zapytania

```
SELECT *
FROM person2
    WITH ( INDEX ( person_first_last_name_idx ))
WHERE lastname = 'Price'
```

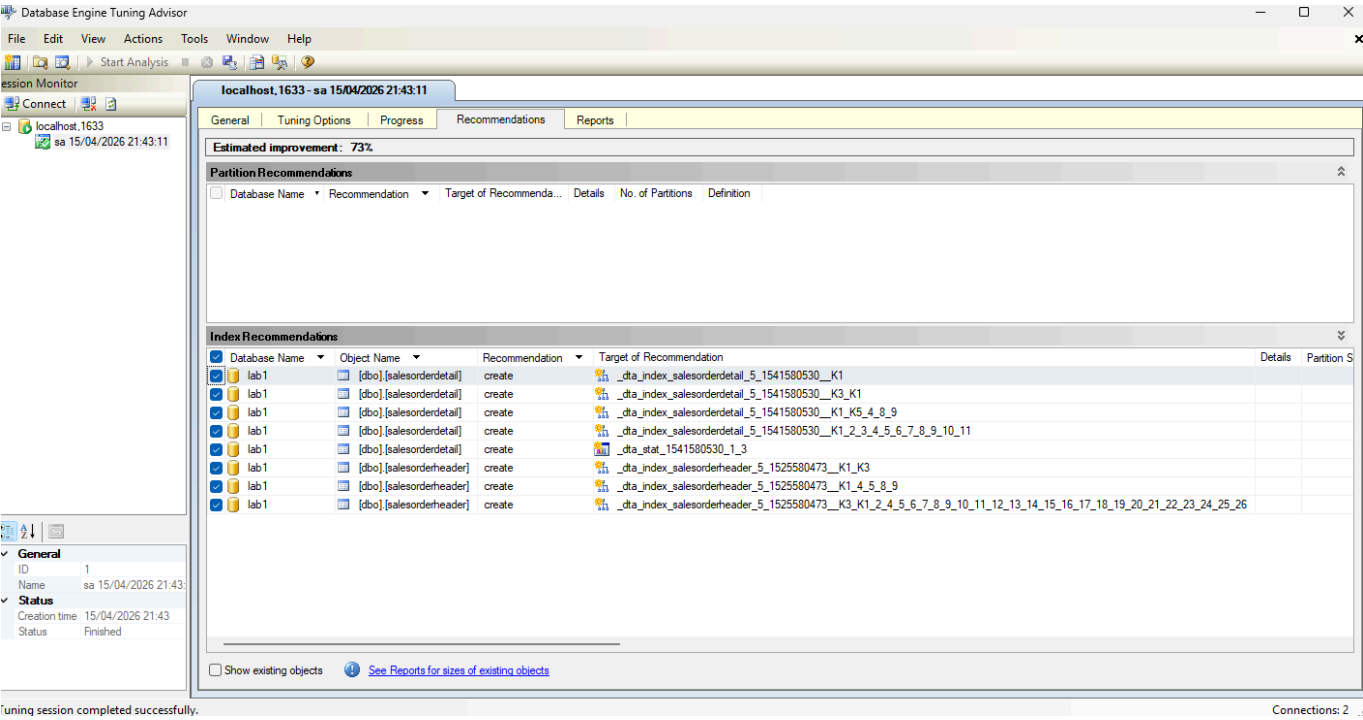
wskazuje na wysoki koszt związany z zwracaniem wierszy przyporządkowanych danym wartościom klucza lastname.

Punktacja:

| zadanie | pkt |
|---------|-----|
| 1       | 2   |
| 2       | 2   |
| 3       | 2   |
| 4       | 2   |
| 5       | 2   |
| razem   | 10  |

Referencje

Rysunek 1 - rekomendacje DTA



Rysunek 2 - blank reports

localhost,1633 - sa 15/04/2026 21:43:11

General

Tuning Options

Progress

Recommendations

Reports

Tuning Summary

Date

15/04/2026

Time

21:51:56

Server

localhost,1633

Database(s) to tune

[lab1]

Workload

Inline

Maximum tuning time

51 Minutes

Time taken for tuning

1 Minute

Estimated percentage improvement

73.14

Maximum space for recommendation (MB)

54

Space used currently (MB)

20

Space used by recommendation (MB)

49

Number of events in workload

4

Number of events tuned

4

Number of indexes recommended to be created

7

Number of statistics recommended to be created

1

Tuning Reports

Select report:

Statement cost report

Statement Id

Statement String

Percent Improvement

Statement Type

y.

Connections: 3